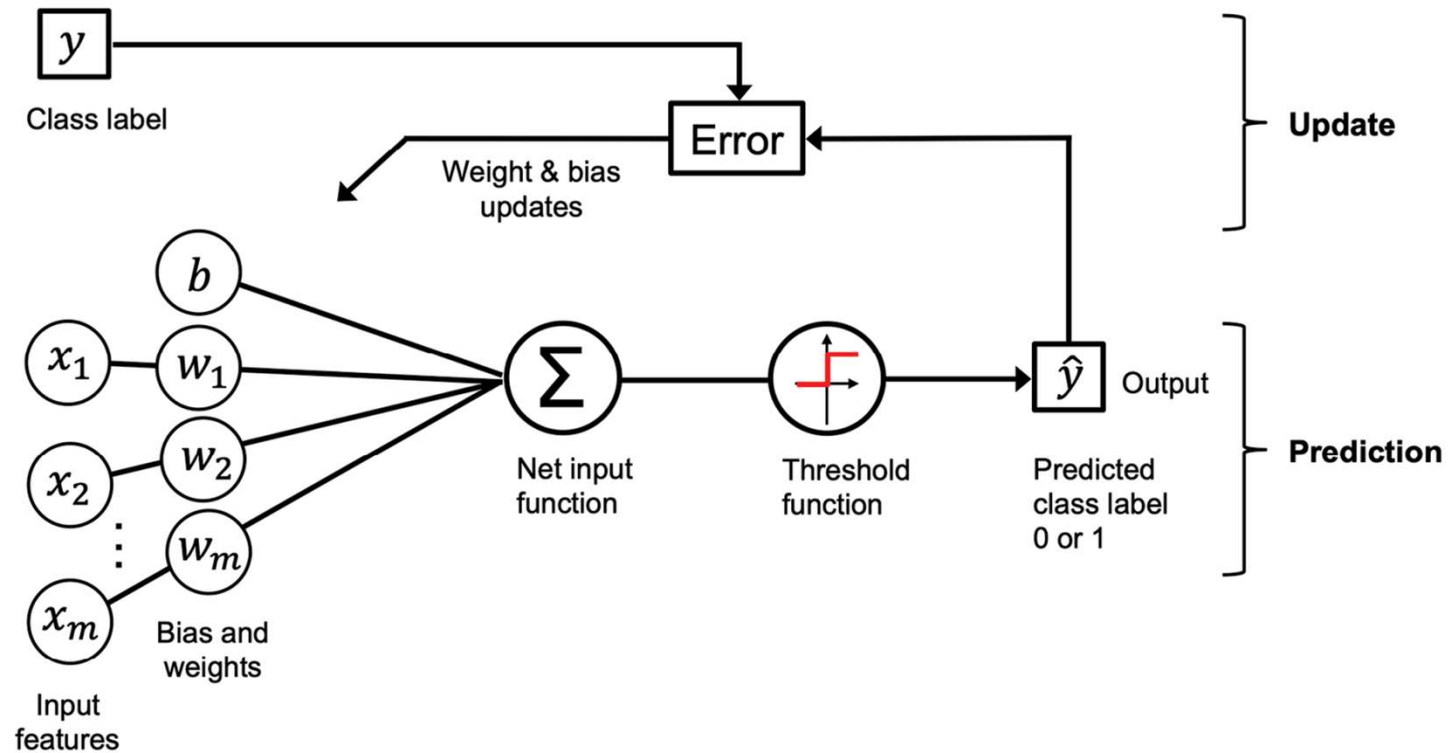


Lecture 24: Deep Convolutional Neural Networks

Sergei V. Kalinin

Training Linear Neuron



From S. Raschka, Machine Learning with PyTorch and Scikit-Learn

Training Linear Neuron

- Initialize the weights and bias unit to 0 or small random numbers
- For each training example, $\mathbf{x}(i)$:
- Compute the output value, $\mathbf{y}(i) = \mathbf{w}^T \mathbf{x}(i) + b$
- Update the weights and bias unit: $w_j := w_j + \Delta w_j$ and $b := b + \Delta b$
- Where $\Delta w_j = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$ and $\Delta b = \eta(y^{(i)} - \hat{y}^{(i)})$

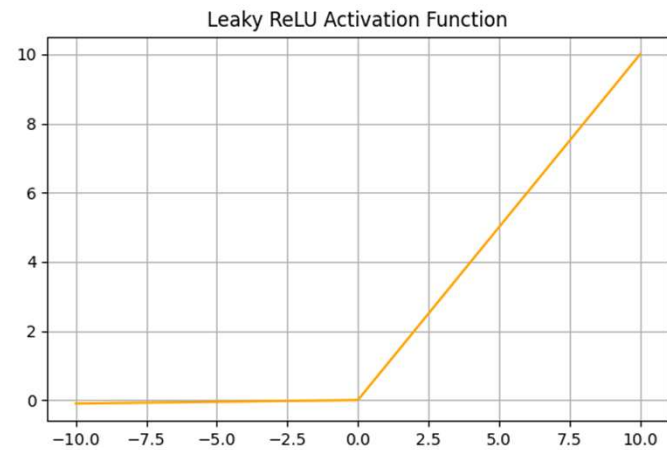
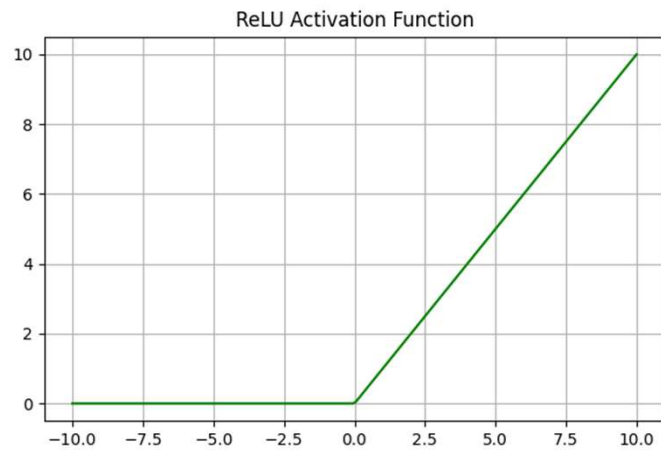
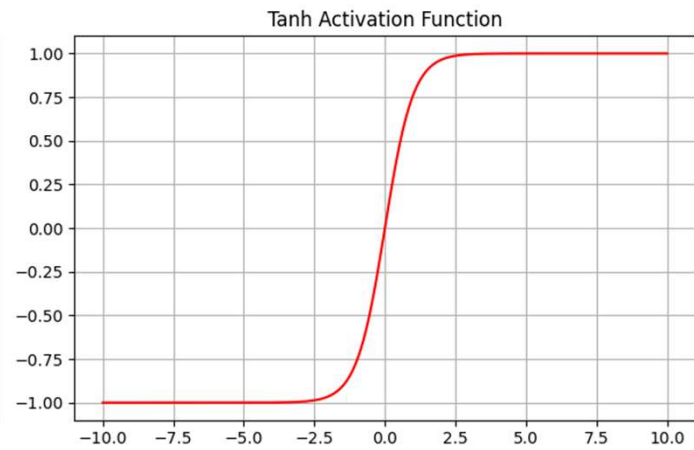
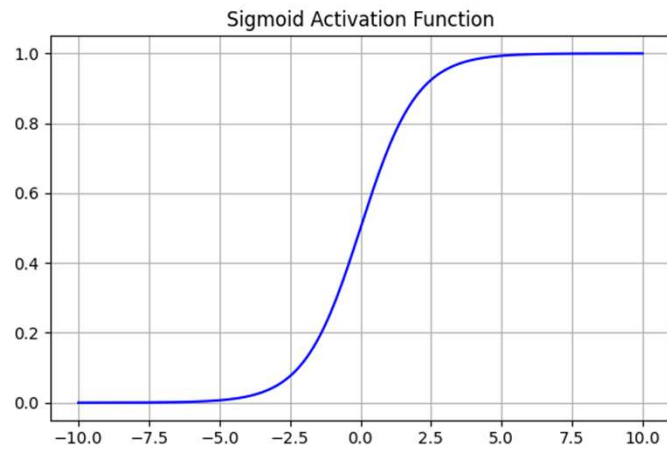
Each weight, w_j , corresponds to a feature, x_j , in the dataset,

η is the **learning rate** (typically a constant between 0.0 and 1.0),

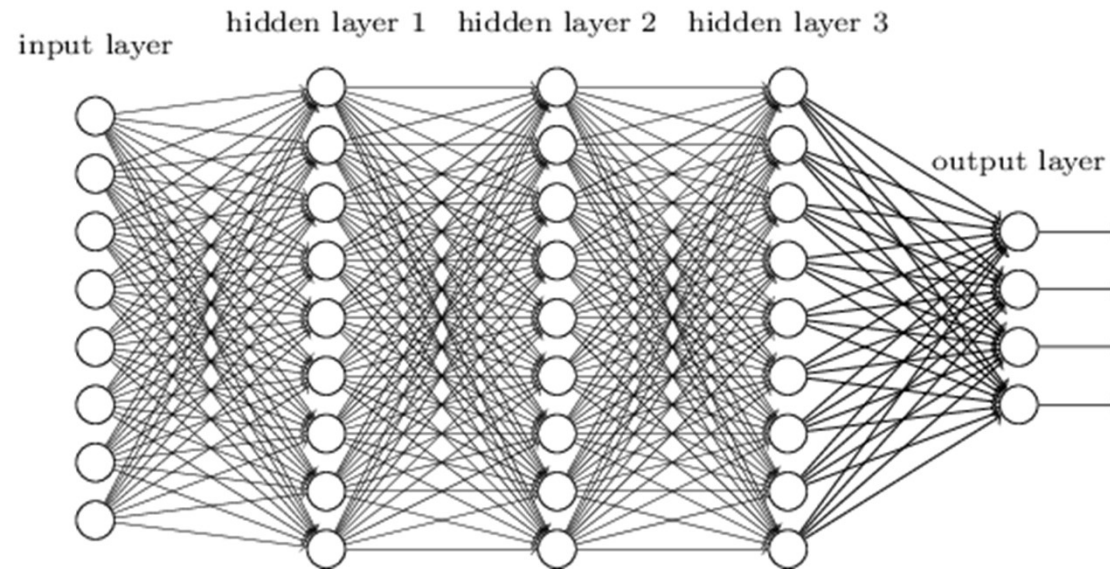
$y^{(i)}$ is the **true class label** of the i th training example,

$\hat{y}^{(i)}$ is the **predicted class label**

Activation functions



Putting Neurons Together



- Linear neurons and activation functions
- Input dimensionality
- Output dimensionality
- Special attention: activation function of the last layer
- Loss function – how different is what we get from what we want (supervised ML)
- Backpropagation – how we adapt the neuron settings given the loss
- Metrics – how we monitor the training/performance

Loss functions for supervised ML

A loss function, also known as a cost function, quantifies the difference between the predicted values and the actual target values. It guides the training of neural networks by providing a measure to minimize during optimization

Mean Squared Error (MSE): Used for regression problems.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Measures the average squared difference between actual and predicted values

Cross-Entropy Loss: Used for classification problems.

$$CE = - \sum_{i=1}^N y_i \log(\hat{y}_i)$$

Measures the performance of a classification model whose output is a probability value between 0 and 1

- Loss functions provide the primary feedback signal for learning.
- The choice of loss function can significantly affect the model's performance and convergence

Loss functions for classifiers

- 1. Binary Cross-Entropy:** Measures the performance of a classification model whose output is a probability between 0 and 1. It's suitable for models where each instance is expected to belong to one of two classes (**Sigmoid**)
- 2. Categorical Cross-Entropy:** Used when there are two or more label classes. Assumes a single-label, multi-class classification problem where each class is mutually exclusive (**Softmax**)
- 3. Sparse Categorical Cross-Entropy:** Similar to CCE but more efficient with a large number of classes. Used when the classes are mutually exclusive and the labels are integers (**Softmax**)
- 4. Hinge Loss:** Commonly used with Support Vector Machine (SVM) classifiers but applicable to neural networks for binary classification tasks. It's designed to create a margin between data points (**Linear** or **Tanh**)
- 5. Focal Loss:** An extension of cross-entropy loss, adding a focusing parameter to balance the importance of correctly classifying different classes, particularly useful in imbalanced datasets. **Sigmoid** (Binary), **Softmax** (Multi-class)
- 6. Kullback-Leibler Divergence:** Measures how one probability distribution diverges from a second, expected probability distribution. Used in scenarios like learning a complex multi-class classification problem or training a model to output a probability distribution (**Softmax**)

Loss functions for regressors

1. Mean Squared Error (MSE): Standard Regression

Calculates the average squared difference between the estimated values and the actual value. Commonly used in regression problems and is sensitive to outliers (Linear).

2. Mean Absolute Error (MAE): Standard Regression

Computes the average of the absolute differences between predictions and actual values. Less sensitive to outliers compared to MSE (Linear)

3. Huber Loss: Robust Regression

Less sensitive to outliers than the MSE. It's quadratic for small errors and linear for large errors, combining the best properties of MSE and MAE (Linear)

4. Quantile Loss: Quantile Regression

Used in scenarios where the goal is to predict an interval instead of only point estimates. It allows the model to learn to predict a range that contains the true value with a certain probability (Linear).

7. Poisson Loss: Count Data Regression

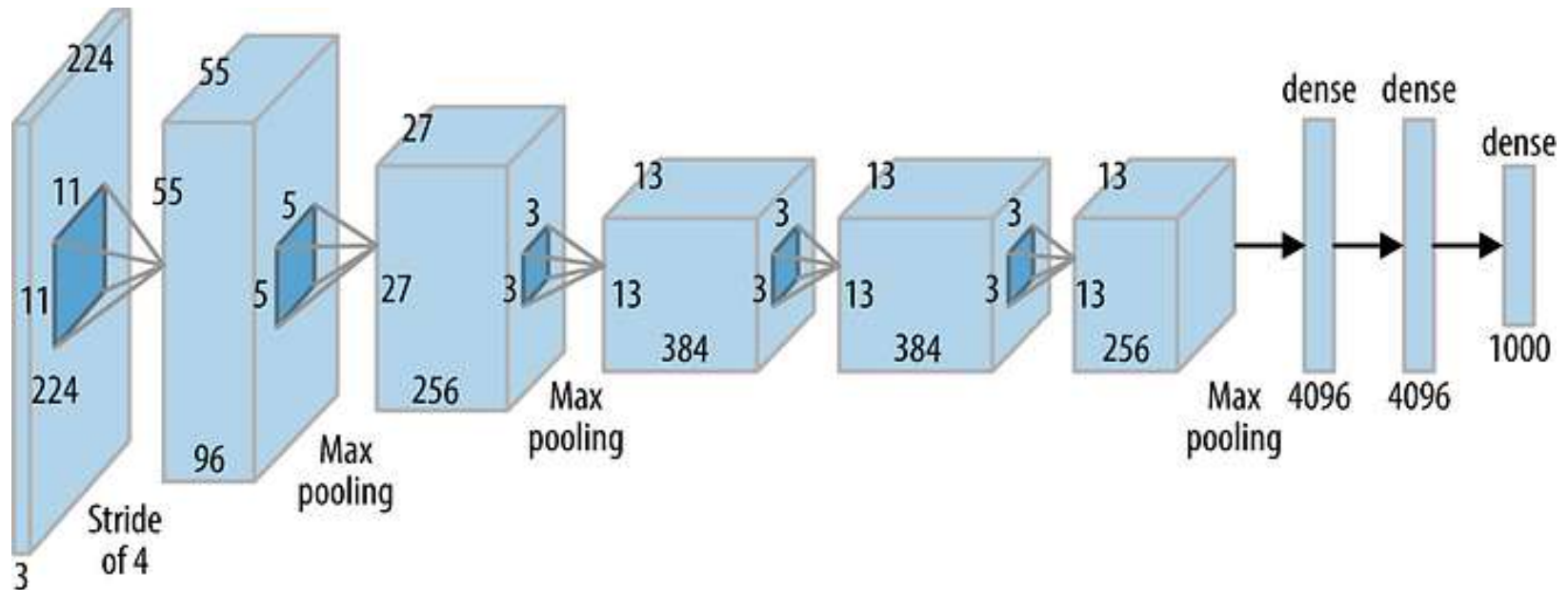
Suitable for modeling count data and intensity rates where the output values are non-negative integers (Exponential)

Loss functions for image segmentation

- 1. Binary Cross-Entropy:** Measures the pixel-wise binary classification loss. Commonly used in binary segmentation tasks (e.g., object vs. background). Ideal for problems where each pixel is classified independently into two classes.
- 2. Categorical Cross-Entropy:** An extension of binary cross-entropy for multi-class segmentation tasks. It calculates the loss for each pixel and classifies it into one of the multiple classes. Used in scenarios with multiple classes (e.g., different types of objects in an image).
- 3. Dice Loss:** Based on the Dice coefficient, which is a measure of overlap between two samples. Particularly effective for data with imbalanced classes, such as medical image segmentation.
- 4. Jaccard/Intersection Over Union (IoU) Loss.** Based on Jaccard index. Useful in evaluating the similarity between the predicted segmentation and the ground truth, especially when the classes are imbalanced.
- 5. Weighted Cross-Entropy:** A variant of cross-entropy loss where each class is given a specific weight, usually inversely proportional to the class frequency. Useful in cases of class imbalance where certain classes need more focus.

Deep Convolutional Neural Networks

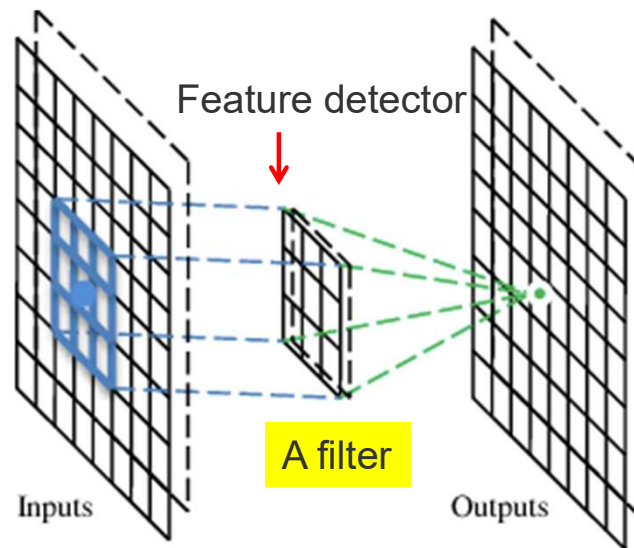
Structure of AlexNet



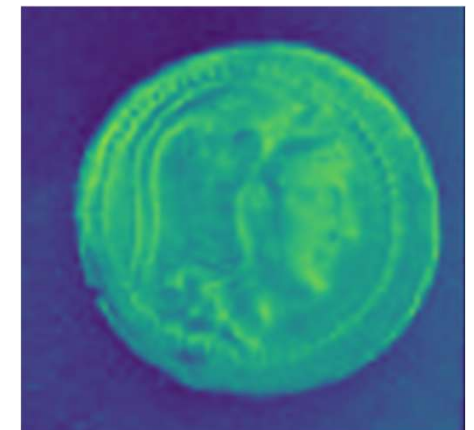
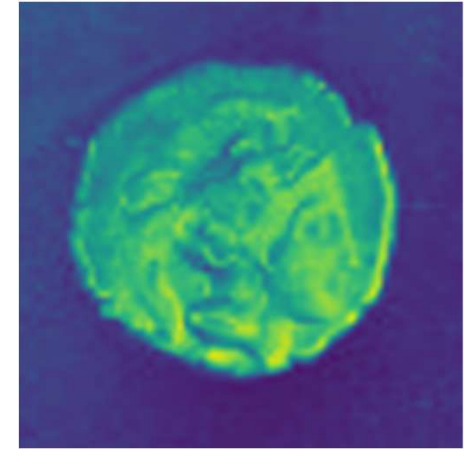
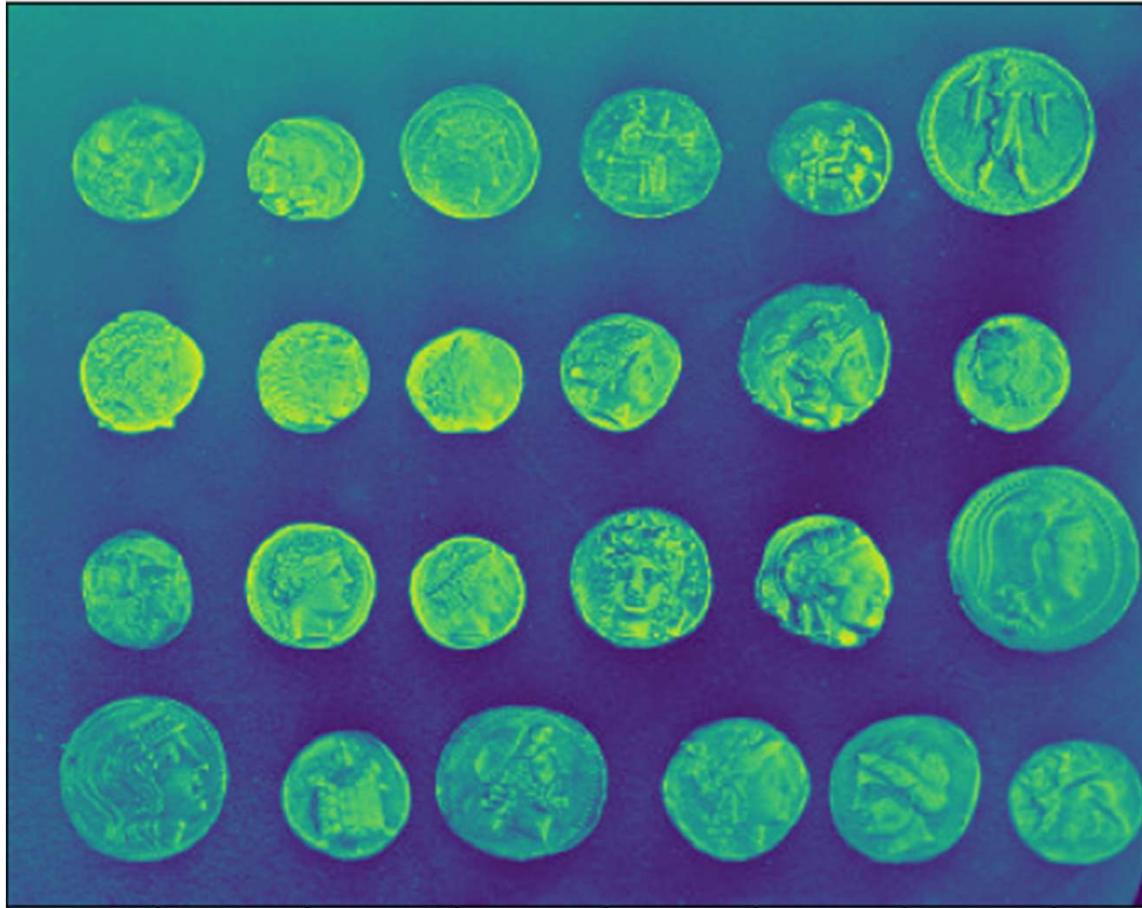
<https://towardsdatascience.com/the-w3h-of-alexnet-vggnet-resnet-and-inception-7baaaecccc96>

Finding features in images

A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.

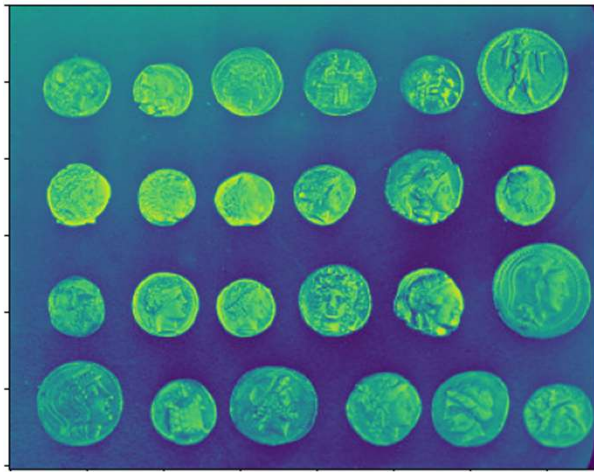


Coin example (scikit-image)

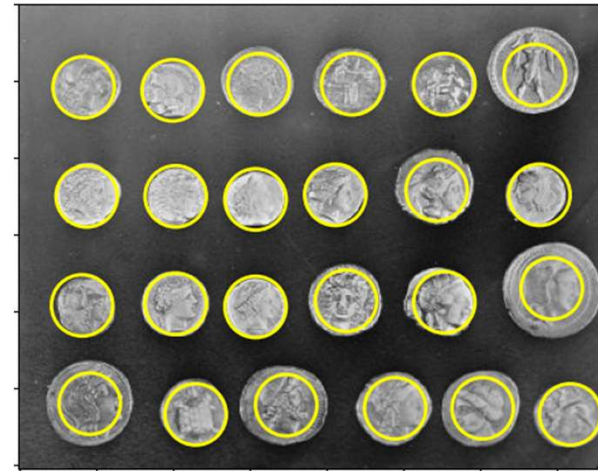


What are features in this image?

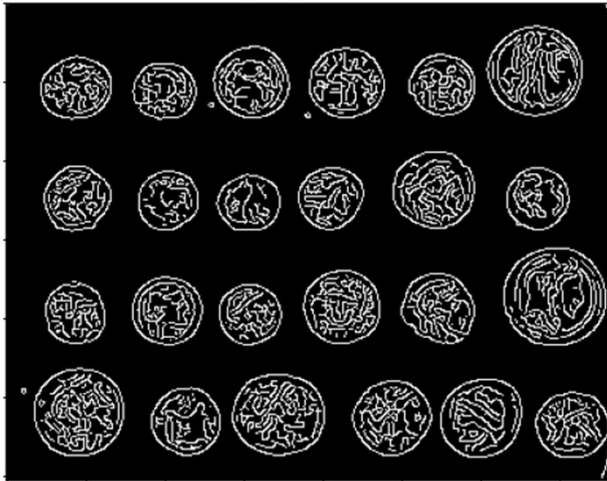
Feature finding in scikit-image



LoG Blob Detection



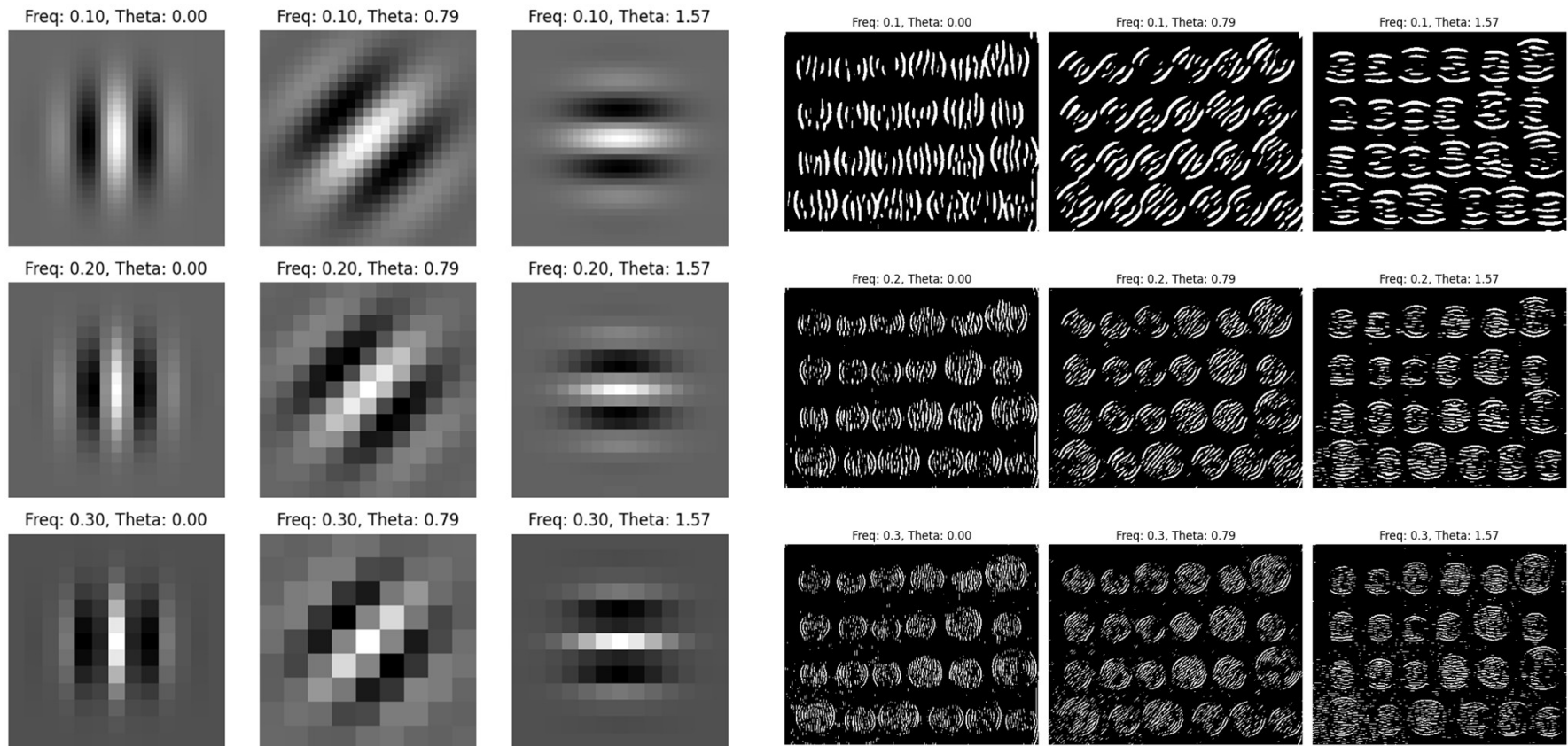
Canny Edge Detection



Hough Circle Transform



Gabor filters



What is convolution?

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

These are the network parameters to be learned.

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

What is convolution?

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot
product

3

-1

6 x 6 image

What is convolution?

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



What is convolution?

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

What is convolution?

-1	1	-1
-1	1	-1
-1	1	-1

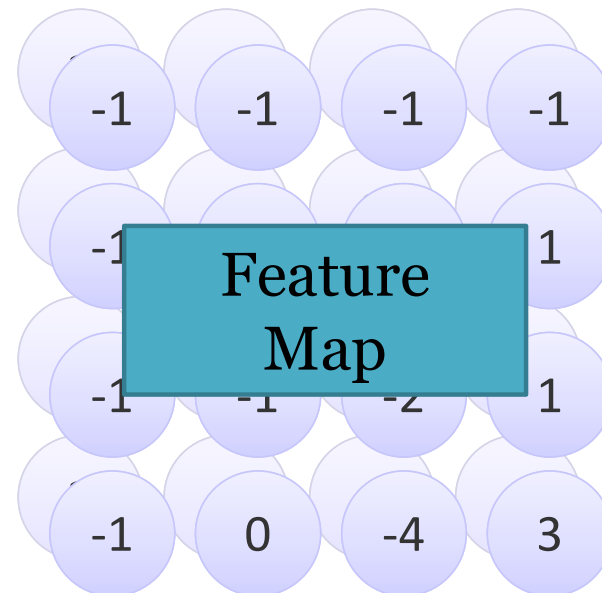
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

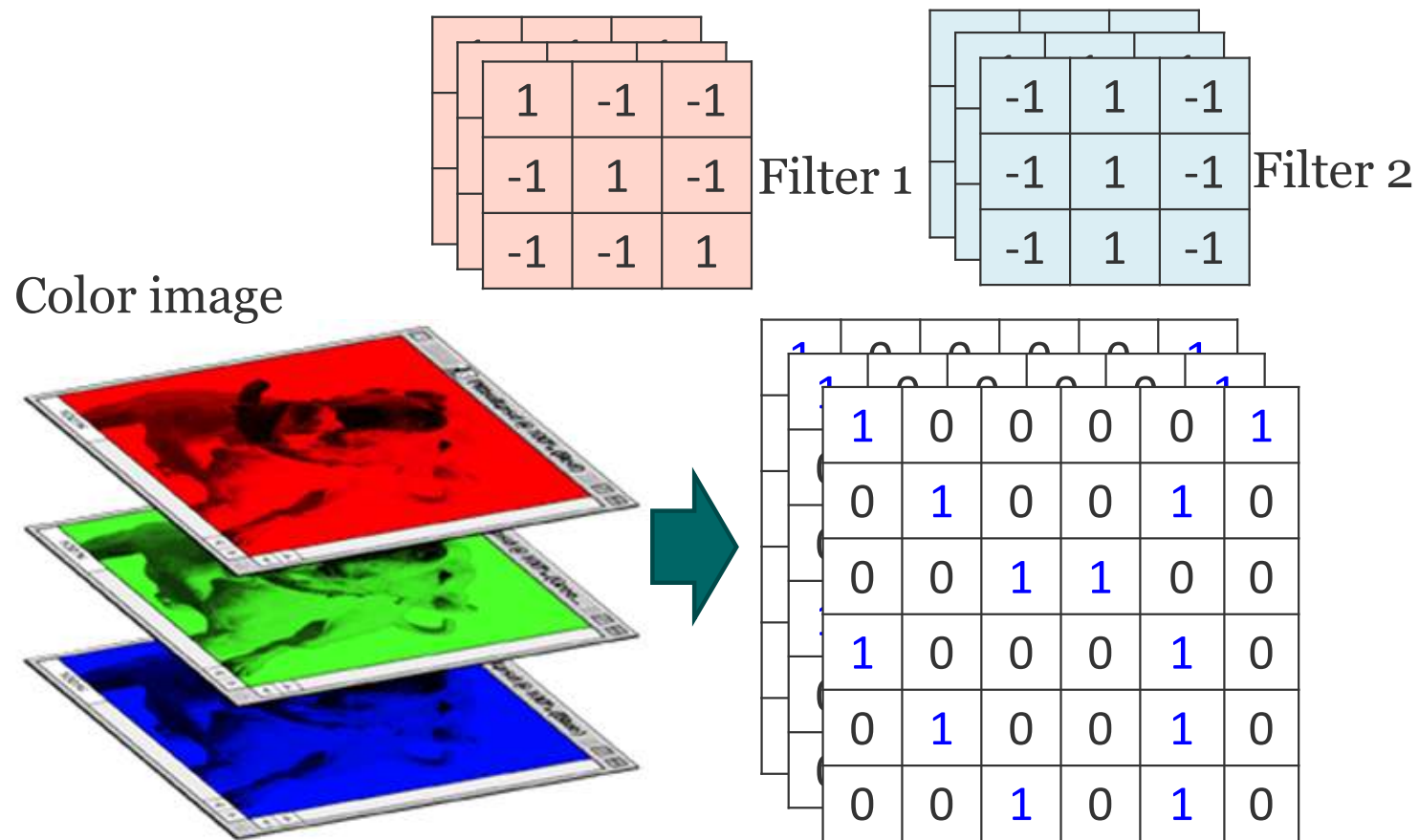
6 x 6 image

Repeat this for each filter

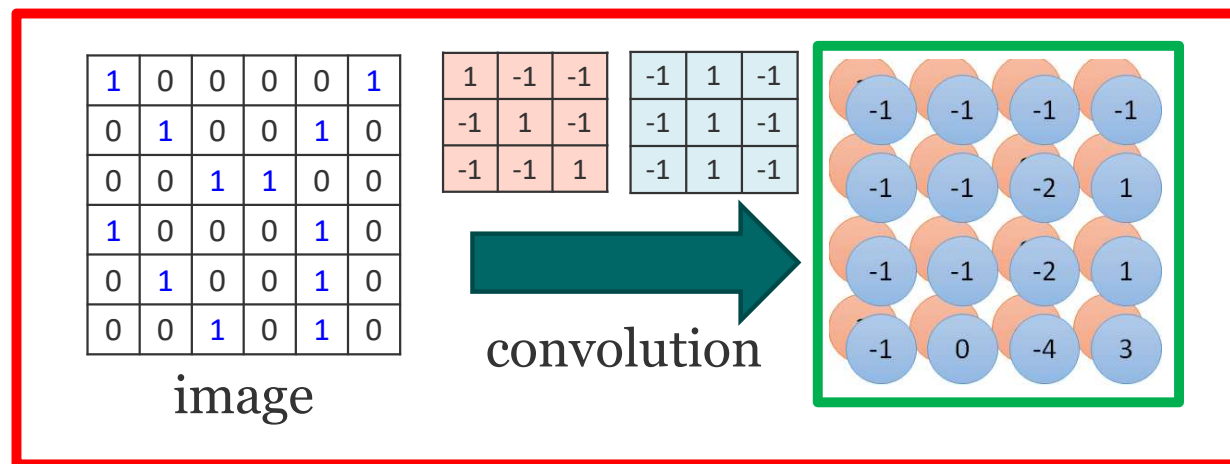


Two 4 x 4 images
Forming 2 x 4 x 4 matrix

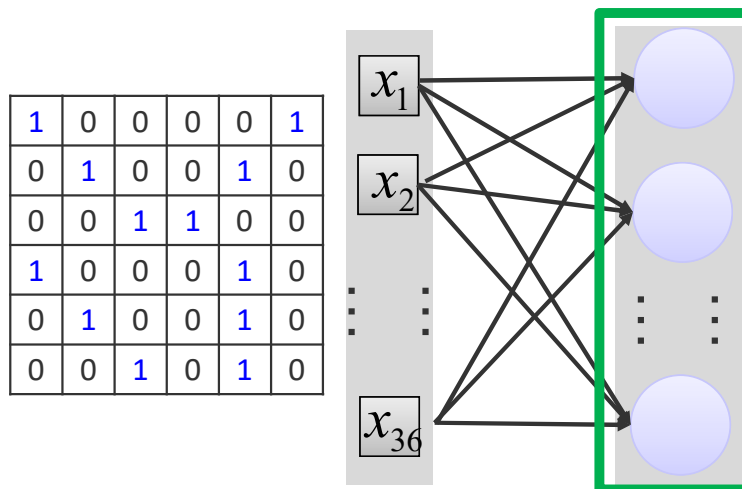
But what about color?

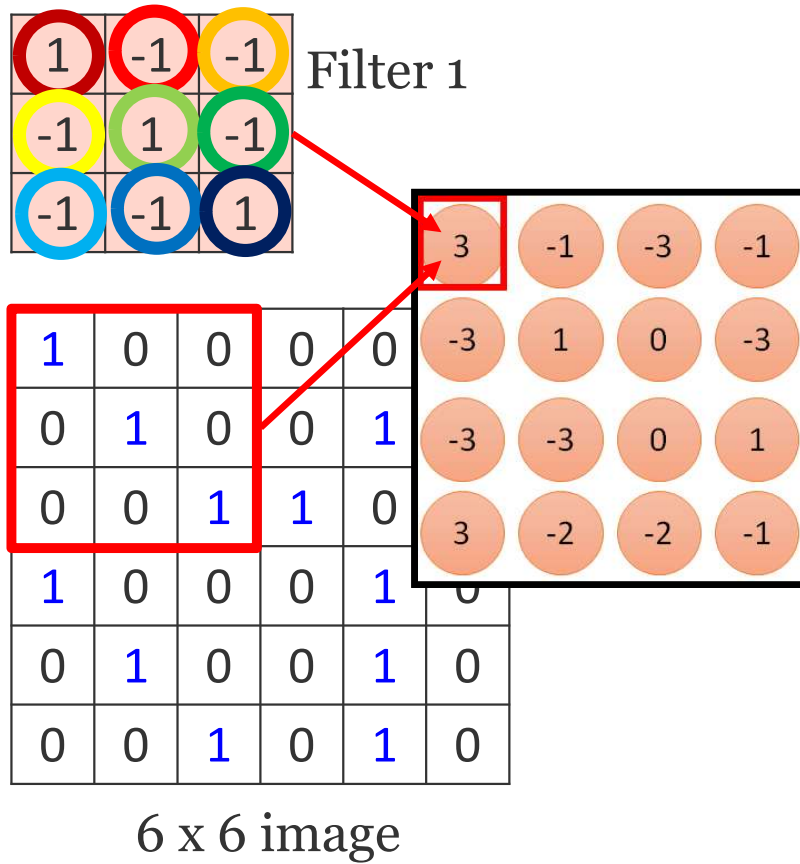


Convolution vs. fully connected

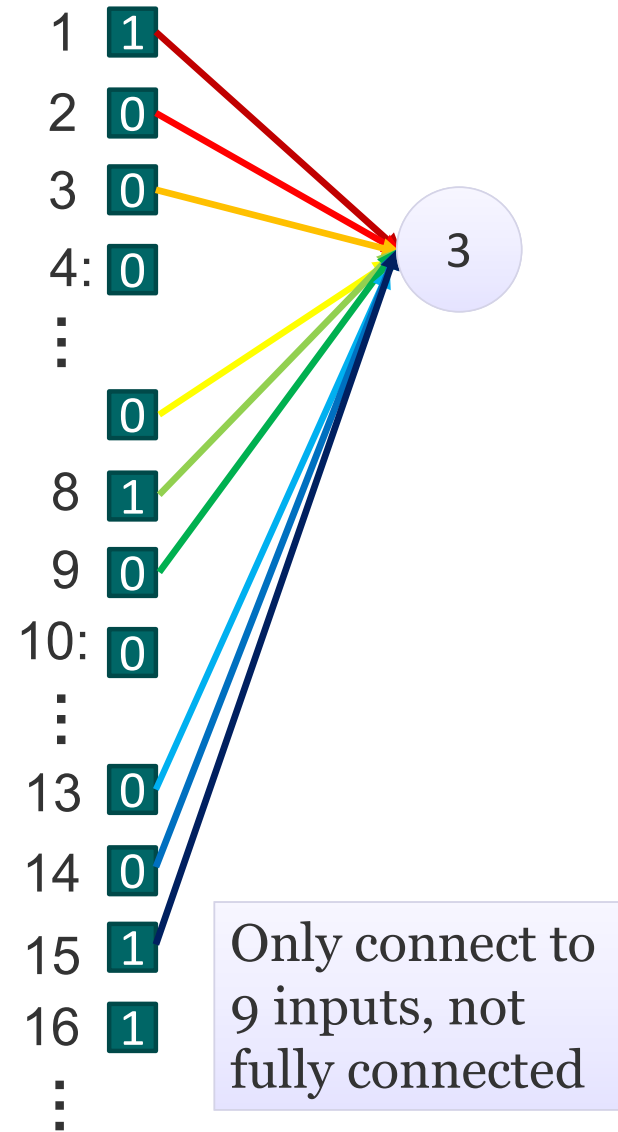


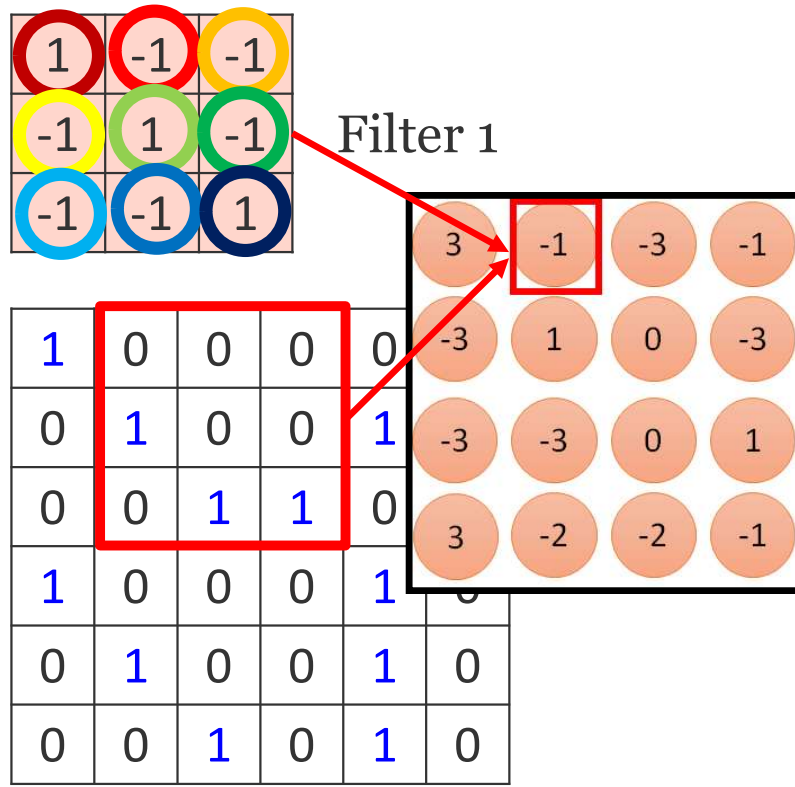
Fully-
connected





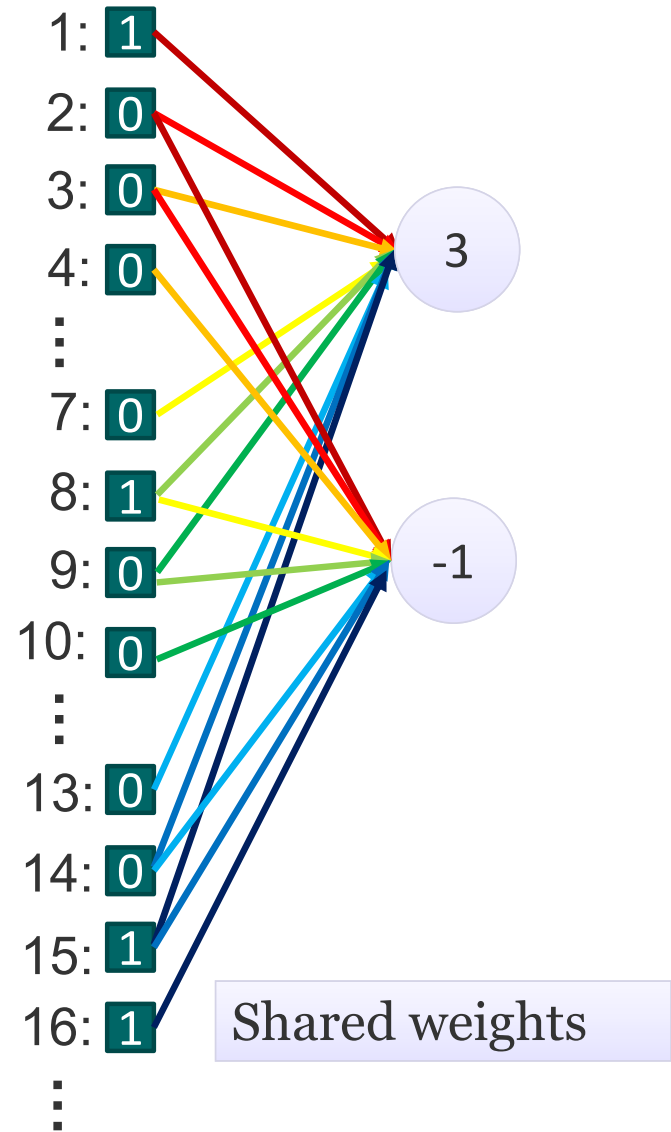
Fewer parameters!





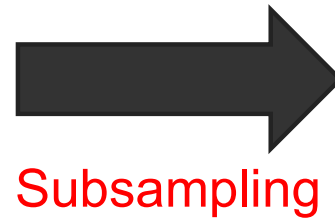
6 x 6 image

Fewer parameters



Scale invariance: pooling

- Subsampling pixels will not change the object
bird



We can subsample the pixels to make image smaller

➡ fewer parameters to characterize the image

MaxPooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

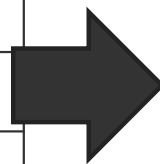
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

MaxPooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



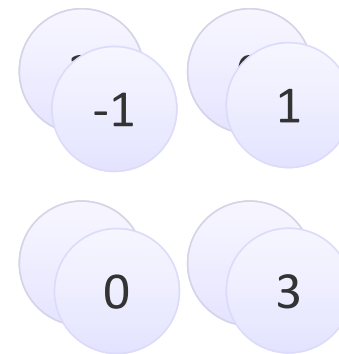
Conv



Max
Pooling



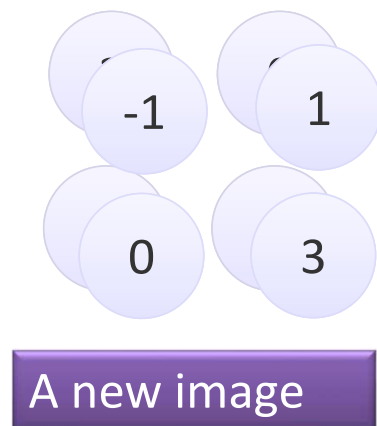
New image
but smaller



2 x 2 image

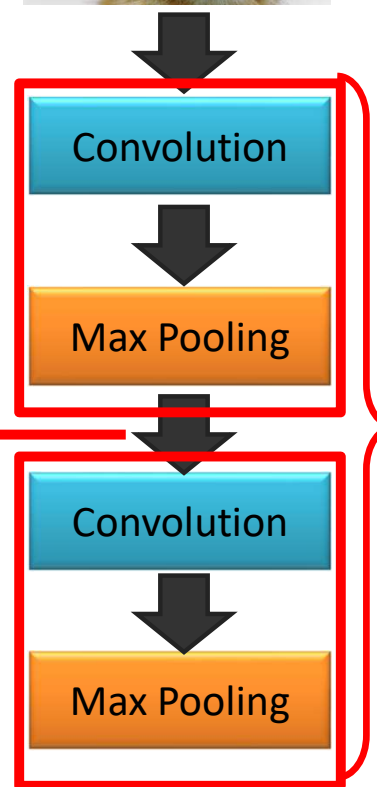
Each filter
is a channel

Making DCNN



Smaller than the original image

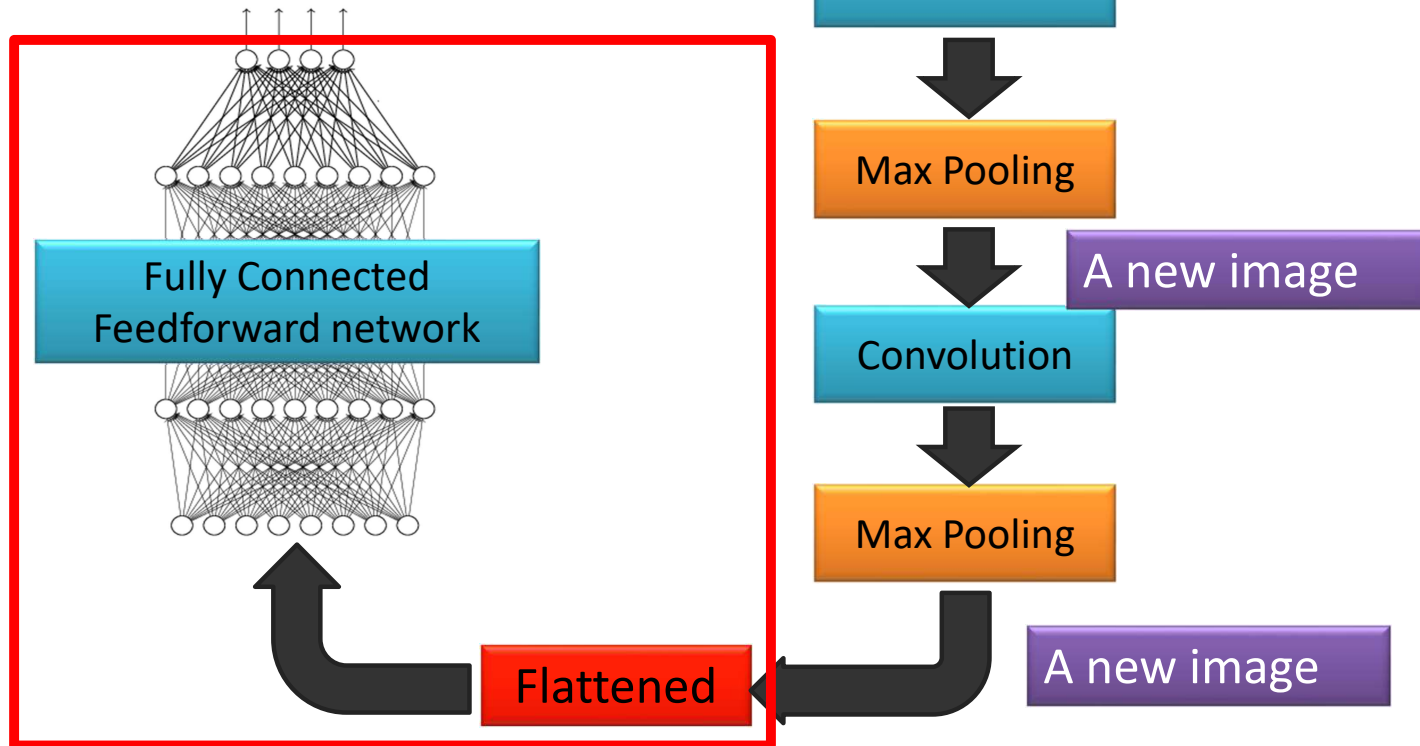
The number of channels is the number of filters



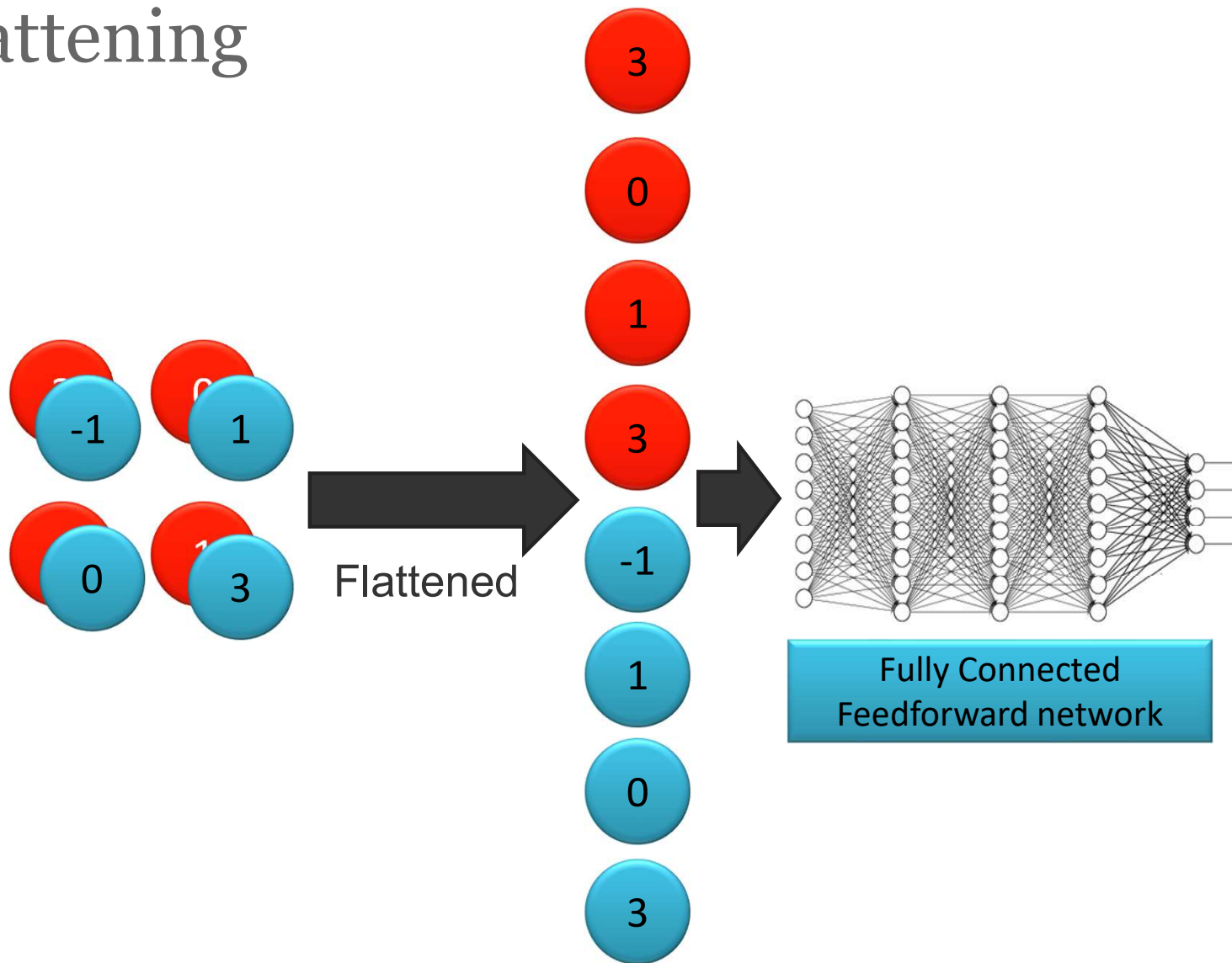
Can repeat many times

Making DCNN

cat dog



Flattening



Keras

Only modified the *network structure* and *input format (vector -> 3-D tensor)*

```
model2.add( Convolution2D( 25, 3, 3,  
                          input_shape=(28, 28, 1)) )
```

1	-1	1
-1	1	-1
-1	-1	-1

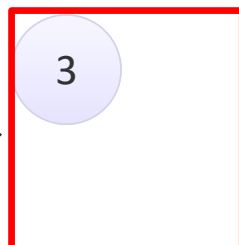
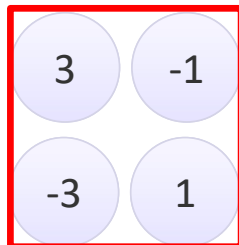
Input_shape = (28 , 28 , 1)

28 x 28 pixels

1: black/white, 3: RGB

There are
25 3x3
filters.

```
model2.add(MaxPooling2D( (2, 2) ))
```



input

Convolution

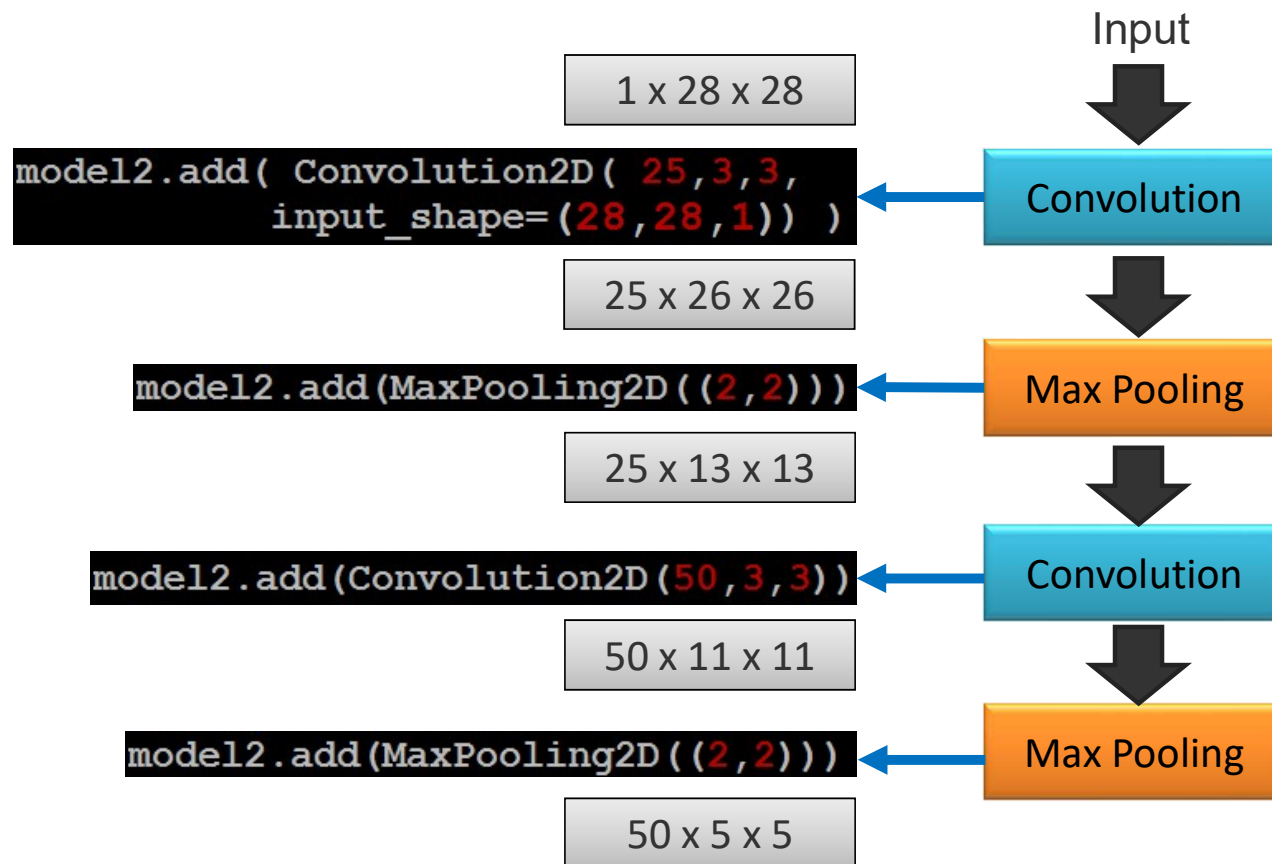
Max Pooling

Convolution

Max Pooling

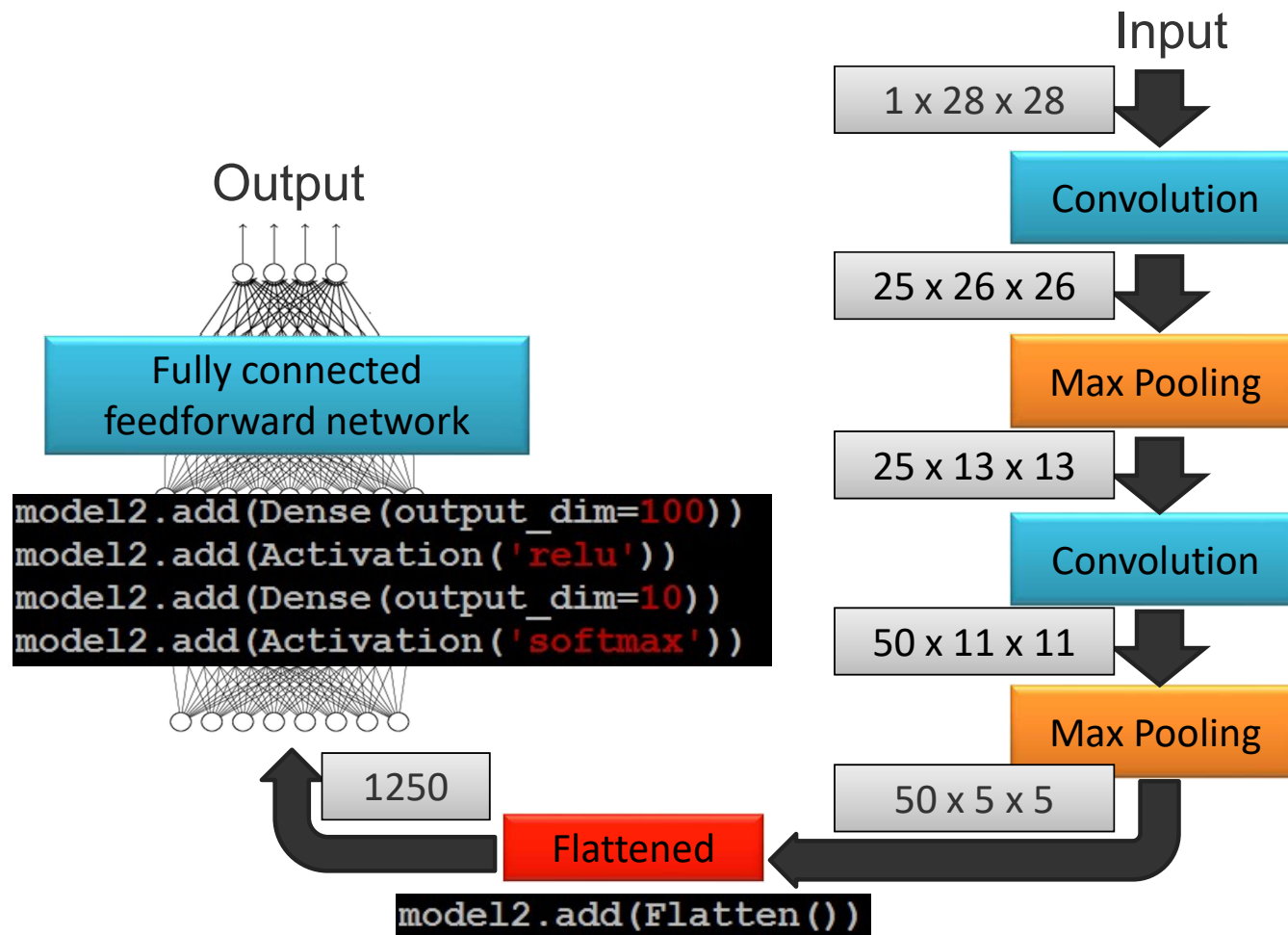
Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*



Keras

Only modified the *network structure* and *input format (vector -> 3-D array)*

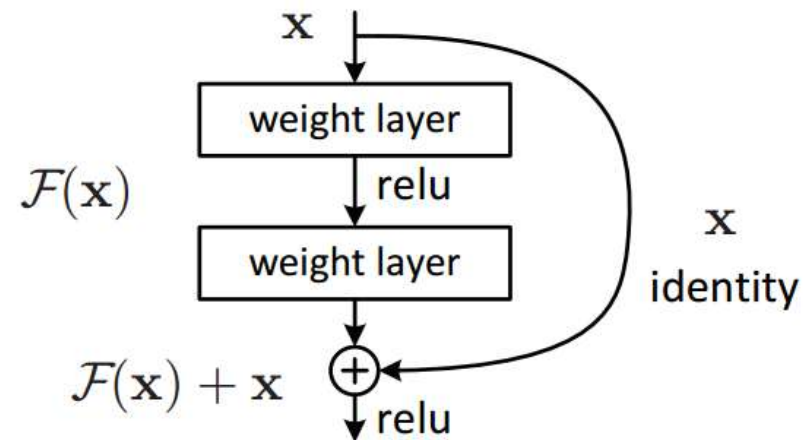


Layer types

1. **Convolutional Layer:** Extracts features from the input image through convolution operations. Key Parameters: Number of filters, kernel size, strides, padding.
2. **Activation Layer:** Introduces non-linearity to the model, allowing it to learn complex patterns. Common Types: ReLU (Rectified Linear Unit), Sigmoid, Tanh.
3. **Pooling Layer:** Reduces the spatial dimensions (height and width) of the input volume, making the model more computationally efficient and less sensitive to the exact location of features. Types: Max Pooling, Average Pooling.
4. **Batch Normalization Layer:** Normalizes the output of a previous layer by subtracting the batch mean and dividing by the batch standard deviation. Helps in speeding up training and reducing the sensitivity to network initialization.
5. **Dropout Layer:** Randomly sets a fraction of input units to zero at each update during training, helping to prevent overfitting.

Layer types

6. **Fully Connected (Dense) Layer:** Neurons in a fully connected layer have connections to all activations in the previous layer, as seen in regular neural networks. Used to classify or regress based on features extracted by the convolutional layers.
7. **Flatten Layer:** Flattens the input from a multi-dimensional tensor to a 1D tensor, making it possible to connect convolutional and pooling layers with dense layers.
8. **Residual Connections** (as in ResNet): Allows the output of one layer to bypass one or more layers and be added to the output of a later layer, improving gradient flow through the network.
9. **Transposed Convolutional Layer (Deconvolution):** Used in generative models and some segmentation tasks; it upscales the input feature map.



DCNNs vs. MLP

1. Parameter Efficiency

- **Fewer Parameters:** CNNs require significantly fewer parameters than FCNs. They use shared weights and convolutional filters, reducing the total number of trainable parameters. This makes CNNs more efficient and less memory-intensive.
- **Reduces Overfitting:** With fewer parameters, CNNs are less prone to overfitting, especially with image data.

2. Exploitation of Spatial Structure

- **Local Connectivity:** CNNs exploit the spatial structure of the data by applying convolutional filters that capture local features (like edges, textures) in early layers and more complex features (like patterns or object parts) in deeper layers.
- **Preservation of Spatial Relationships:** Unlike FCNs that lose spatial relationships by flattening the input, CNNs maintain the spatial hierarchy and relationships between different parts of the input.

3. Translation Invariance

- **Robust to Translation:** Due to pooling layers and the nature of convolution operations, CNNs are inherently more robust to the translation of input data. This means that if an object shifts in an image, a CNN can still detect it effectively.

Hyperparameter tuning

- Hyperparameters are the configuration settings used to structure the neural network and guide the learning process.
- Unlike weights, hyperparameters are not learned from the data but set prior to the training process.
 - **Number of Layers and Neurons:** Too few may lead to underfitting, while too many can cause overfitting and increased computational cost.
 - **Learning Rate:** If it's too high, the model may overshoot minima; if too low, training may be slow or get stuck in local minima.
 - **Batch size:** Influences the accuracy of the gradient estimation and can affect both the speed of convergence and the stability of the learning process.
- How do we do it?
 - Grid Search
 - Random Search
 - Bayesian Optimization
 - **Leveraging Experience:** One can use their knowledge from previous projects to choose a good starting point for hyperparameter settings. This intuition, built from empirical evidence of what has worked in the past, can significantly reduce the search space and lead to faster convergence on optimal or near-optimal configurations.